

# Artificial Intelligence as a Personal Mentor in Frontend Development Education: Methodology and Experimental Results

Nikita Shevtsiv

Department of Virtual Reality Systems  
Institute of Computer Science and  
Information Technologies  
Lviv Polytechnic National University  
Lviv, Ukraine  
nikita0503ua@gmail.com

Olha Kliuchak

Department of Virtual Reality Systems  
Institute of Computer Science and  
Information Technologies  
Lviv Polytechnic National University  
Lviv, Ukraine  
olya22052005@gmail.com

**Abstract**—This paper investigates the use of large language models (LLMs) as personal mentors for students learning frontend development in higher education institutions. We propose a methodology based on the cyclical model “theory – practice – reflection” with AI support at each stage, where AI generates personalized learning projects, provides contextual guidance within the student’s development environment, and delivers feedback without providing ready-made solutions. An experimental evaluation with a group of 5 students demonstrated high effectiveness of the approach: the AI mentor received the highest usefulness rating (4.8 out of 5) among all components of the learning process. The results confirm the feasibility of integrating AI mentorship into computer science education as a scalable, cost-free, and always-available complement to traditional teaching methods.

**Index Terms**—artificial intelligence, mentoring, frontend development, programming education, large language models, higher education

## I. INTRODUCTION

The modern IT labor market demands that frontend developers possess not only knowledge of programming language syntax but also understanding of architectural patterns, experience with real APIs, version control systems, and the ability to independently solve complex technical problems. According to the Stack Overflow Developer Survey 2025, 44% of developers used AI-enabled tools for learning new technologies, an increase of 7 percentage points from the previous year [1].

Traditional higher education does not always provide the required level of practical preparation. Lecture formats build theoretical foundations but do not develop skills for independent problem-solving in real-world contexts. Laboratory assignments are typically isolated exercises that do not reflect the complexity of commercial development. Online courses offer structured learning but often reduce to passive repetition of instructor actions.

Classical mentorship is recognized as one of the most effective methods for learning programming [2]. An experienced mentor adapts explanations to the student’s level, provides

personalized feedback, and guides toward correct solutions without providing ready-made answers. However, mentorship is expensive, limited in time, and does not scale.

The development of large language models (LLMs) opens fundamentally new possibilities for addressing this problem. Modern AI systems can perform key mentor functions: explaining concepts, generating learning tasks, analyzing code, and preparing students for technical interviews — while remaining free (within free-tier limits), available 24/7, and capable of adapting to the context of the student’s specific project. Research published in Nature (2025) confirms that students learning with AI tutors acquire material faster and demonstrate higher motivation compared to traditional formats [3].

The objective of this paper is to propose and validate a methodology for using AI as a personal mentor in frontend development education.

## II. PROBLEM STATEMENT

Analysis of junior frontend developer vacancies reveals typical employer requirements: proficiency in JavaScript/TypeScript, experience with React (hooks, state management, routing), understanding of REST APIs, Git workflow, and ability to work with existing codebases. Comparison of these requirements with typical university curricula reveals a significant gap: academic courses focus on fundamental disciplines that form a necessary foundation but do not provide practical experience with modern frameworks and tools.

### A. Limitations of Existing Learning Formats

Existing learning formats have notable limitations that prevent effective preparation of frontend developers:

**Lectures and laboratory work** provide structured material delivery but have a fixed pace and do not adapt to individual student levels. A student who does not understand a concept on the first attempt has no opportunity to receive a repeated explanation from a different angle. Laboratory assignments

are typically isolated exercises — implementing a single component or function — that do not reflect the complexity of real-world projects where multiple systems interact.

**Online courses** predominantly follow the “repeat after instructor” format — students reproduce code from video tutorials but do not develop independent problem-solving skills. When encountering a problem not covered by the course, students are left without support. A meta-analysis of AI tools in programming education [6] confirms that traditional online formats show limited effectiveness compared to AI-assisted approaches.

**Self-directed learning from documentation** requires high self-organization and prior experience that beginners typically lack. A novice often does not know what to search for, does not understand the context of what they read, and receives no feedback on the quality of their solution.

### B. Mentorship: Effective but Not Scalable

Individual mentorship addresses these limitations effectively [2] by adapting explanations, providing code review, guiding without giving direct answers, and supporting motivation. Research consistently shows that personalized tutoring produces significant learning gains [3], [8].

However, mentorship remains financially expensive, time-limited (mentor available only at scheduled hours), subjective (dependent on specific mentor competence), and non-reproducible (each session is unique). A single mentor physically cannot support dozens of students simultaneously, making this approach inherently non-scalable for university education.

These constraints create a demand for an approach that combines the personalization and adaptiveness of mentorship with the scalability and accessibility of digital tools.

## III. AI AS A MENTOR: CAPABILITIES AND LIMITATIONS

Modern LLMs have demonstrated the capability to effectively perform functions traditionally belonging to mentor competence [4], [5]:

- 1) **Concept explanation.** AI can explain any frontend development concept adaptively — from basic to advanced level, with analogies and repeated explanations from different angles. Unlike a human mentor, AI does not tire and is ready to explain the same concept repeatedly without frustration.
- 2) **Task generation.** AI can create practical tasks of appropriate difficulty with detailed acceptance criteria, enabling students to self-evaluate results.
- 3) **Code analysis and feedback.** AI analyzes student-written code, identifies potential problems and suggests improvement directions — without providing ready-made code.
- 4) **Interview preparation.** AI models the format of technical interviews: asks typical questions, analyzes student responses, and identifies gaps in understanding.

- 5) **Architectural consulting.** When working on projects, AI helps make architectural decisions: file structure, component responsibility distribution, library selection.

**Key advantages** of AI mentorship include: zero cost (free-tier LLM plans are sufficient for learning purposes), 24/7 availability, unlimited patience, personalization through project context (in smart IDE environments, AI has access to student’s code structure and current errors), and development of independence.

**Limitations** include possible generation of incorrect information (hallucinations), risk of dependency formation, and necessity to verify responses. However, these risks should be evaluated in context: an incorrect AI response costs the student 20–30 minutes of additional debugging, whereas absence of any mentorship costs weeks of stagnation. Moreover, the skill of critically evaluating AI output is itself a valuable professional competency [1].

**Requirements for the student:** self-direction, desire to understand, discipline, critical thinking, and initiative in formulating specific questions.

## IV. PROPOSED METHODOLOGY

The proposed methodology is based on a cyclical model: **theory** → **practice** → **reflection**, where AI provides support at each stage (Table I).

TABLE I  
CYCLICAL LEARNING MODEL WITH AI MENTOR

| Stage      | Student Activity                    | AI Mentor Role                                  |
|------------|-------------------------------------|-------------------------------------------------|
| Theory     | Studies new concept, asks questions | Explains, provides analogies, adapts complexity |
| Practice   | Writes code independently           | Consults, gives hints, does NOT write code      |
| Reflection | Presents result for analysis        | Provides feedback, suggests improvements        |

The cycle repeats for each new concept or sprint.

### A. AI as Learning Environment Generator

The central element of the methodology is that AI not only mentors but also **generates the personalized learning project** for the student:

- 1) The student selects a project domain (“task manager,” “online store,” “blog platform”) or asks AI to propose a project of appropriate complexity.
- 2) AI designs the architecture: defines pages, components, API endpoints, data model.
- 3) AI divides the project into sprints with progressive complexity and creates structured tasks with detailed acceptance scenarios.
- 4) The student executes tasks sequentially; AI mentors during execution.
- 5) After each task completion, AI analyzes the result and provides feedback.

This approach simulates a real IT company workflow: a developer receives a task with requirement description, implements it independently, and undergoes code review.

### B. Contextualization in Smart IDE

The AI mentor operates directly within the student’s development environment (smart IDE). It has access to: project structure, technology stack and configuration, current student code, and current sprint task descriptions. This enables contextual responses rather than abstract documentation examples.

For example, when a student asks “How do I implement a protected route?” the AI responds considering the specific router structure in the student’s project, the libraries already in use, and the existing authorization code — rather than providing a generic example from documentation. This contextualization is what distinguishes AI mentorship from general-purpose chatbots: the mentor understands the student’s specific situation.

### C. Interview Preparation and Pet Projects

AI effectively models technical interview format for frontend developers. This includes:

- **Theoretical questions:** explaining event loop, Virtual DOM, closures, React component lifecycle;
- **Practical tasks:** implementing debounce functions, custom hooks, state management patterns;
- **Architectural discussions:** choosing between Context API and Redux, structuring large applications.

AI analyzes the student’s responses, identifies inaccuracies, and suggests how to improve the answer — simulating a real interviewer in a dialogical format.

For pet projects, AI serves as an architectural advisor and reviewer — helping choose technologies, reviewing project structure, analyzing written code, and suggesting new features. The result is a portfolio project written independently by the student, with understanding of every line of code. The key principle throughout: AI explains concepts and indicates direction but does not generate ready-made code for copying.

## V. EXPERIMENTAL EVALUATION

### A. Experimental Setup

To validate the proposed methodology, a learning platform was developed implementing the described principles. The platform represents a full-featured software project — a task manager built on a modern technology stack (React, TypeScript, Redux Toolkit, REST API).

The project is organized using a sprint model with progressive complexity:

- **Sprint 1** — Authorization: registration, login, JWT tokens, protected routes;
- **Sprint 2** — Data reading: list display, user profile, error handling, loading states;
- **Sprint 3** — Full CRUD: create, edit, delete tasks, file handling, pagination;
- **Sprint 4** — Extended functionality: priorities, redesign, additional filters.

Each sprint contains structured tasks with detailed acceptance scenarios (happy path, error handling, edge cases), simulating the format of commercial development work.

A pilot evaluation was conducted with a group of 5 students. Participant profile: basic knowledge of JavaScript and React, no commercial development experience. Participants completed platform tasks (sprints 1–4) using an AI mentor in a smart IDE over a 4-week period.

### B. Results

Evaluation was performed across four criteria (Table II).

TABLE II  
PILOT TESTING RESULTS (N=5)

| Criterion                            | Mean | Min | Max |
|--------------------------------------|------|-----|-----|
| Sprint 1 completion time (days)      | 4.2  | 3   | 6   |
| Task clarity (1–5)                   | 4.4  | 4   | 5   |
| AI mentor usefulness (1–5)           | 4.8  | 4   | 5   |
| Work readiness self-assessment (1–5) | 4.0  | 3   | 5   |

The AI mentor received the highest rating among all criteria (4.8/5), indicating its perception by students as the most valuable element of the learning process.

### C. Qualitative Feedback

Participants noted the following aspects of AI mentor interaction:

- ability to receive explanations at any time without waiting;
- contextual responses — AI understood their project structure;
- absence of psychological pressure — ability to ask basic questions without embarrassment;
- usefulness in error analysis — AI explained error causes in the context of specific files;
- motivation support — AI acknowledged progress and suggested next steps.

Participants also reported that AI mentorship significantly reduced the time spent “stuck” on problems. Previously, encountering an unfamiliar error could block progress for hours until a human mentor became available. With AI, the typical resolution time for common issues dropped to minutes.

Participant recommendations included: adding more visual examples of expected results (UI screenshots), increasing the number of sprints with more complex topics (WebSocket, real-time updates), and providing initial onboarding video for environment setup.

## VI. COMPARISON WITH TRADITIONAL APPROACHES

Table III compares AI mentorship with traditional approaches to programming education across key criteria.

The comparison demonstrates that AI mentorship combines the key advantages of human mentorship (personalization, high-quality feedback, project context) with the advantages of digital platforms (scalability, availability, zero cost).

TABLE III  
COMPARISON OF APPROACHES TO PROGRAMMING EDUCATION

| Criterion       | Lect.   | Online Courses | Human Mentor | AI Mentor |
|-----------------|---------|----------------|--------------|-----------|
| Availability    | Sched.  | 24/7           | Appt.        | 24/7      |
| Cost            | Tuition | Free/Paid      | High         | Free      |
| Personalization | Low     | Low            | High         | High      |
| Scalability     | Med.    | High           | Low          | High      |
| Feedback        | Limited | Auto           | Quality      | Quality   |
| Adaptiveness    | Low     | None           | High         | High      |
| Patience        | Limited | —              | Limited      | Unlim.    |
| Project context | No      | No             | Yes          | Yes       |

## VII. DISCUSSION

The experimental results, while preliminary due to the small sample size, reveal several important observations about AI mentorship in programming education.

**Effectiveness across skill levels.** Participants with slightly stronger initial knowledge completed sprints faster but rated AI usefulness equally high. This suggests that AI mentorship benefits students regardless of their starting level — adapting its explanations to each individual.

**The “always available” factor.** Qualitative feedback consistently highlighted availability as a critical advantage. Students reported working at unconventional hours (evenings, weekends) when human mentors would be unavailable. The elimination of waiting time between encountering a problem and receiving guidance appears to significantly reduce frustration and maintain learning momentum.

**Critical thinking development.** An unexpected finding was that students who encountered occasional AI hallucinations (incorrect suggestions) reported developing stronger code verification habits. They learned to cross-reference AI suggestions with documentation — a valuable professional skill that traditional mentorship does not explicitly develop.

**Limitations of the study.** The sample size ( $n=5$ ) limits generalizability. The absence of a control group prevents direct comparison with alternative approaches. Self-reported metrics may be subject to bias. Future research should address these limitations through larger-scale randomized controlled trials.

**Scalability implications.** Unlike human mentorship, AI mentorship has near-zero marginal cost per additional student. A university could provide AI mentor access to hundreds of students simultaneously without additional staffing. This makes the approach particularly relevant for institutions with limited teaching resources.

## VIII. CONCLUSIONS

This paper investigated the use of LLM-based artificial intelligence as a personal mentor for frontend development students. The main contributions are:

- 1) We identified the gap between academic preparation and market requirements for frontend developers, as well as limitations of existing learning formats.
- 2) We theoretically justified the capability of modern LLMs to perform key mentor functions and analyzed advantages, limitations, and requirements for effective use.

- 3) We proposed a methodology based on the “theory – practice – reflection” cycle with AI support at each stage, including AI generation of personalized learning environments.
- 4) We described a practical implementation using a sprint-based learning platform with a contextualized AI mentor in a smart IDE.
- 5) Experimental evaluation confirmed the approach effectiveness: the AI mentor received the highest usefulness rating (4.8 out of 5) among all learning process components.

**Practical implications.** AI mentorship can be integrated into higher education as a complement to existing formats. The instructor’s role transforms from knowledge carrier to learning process curator who monitors progress and verifies results.

**Future work** includes conducting a comparative experiment with a control group, increasing participant sample size, and investigating effectiveness for other development directions.

## ACKNOWLEDGMENT

The authors would like to thank the students who participated in the pilot evaluation for their valuable feedback and engagement throughout the experimental period.

## REFERENCES

- [1] “Stack Overflow Developer Survey 2025,” Stack Overflow, 2025. [Online]. Available: <https://survey.stackoverflow.co/2025/>
- [2] A. Begel and B. Simon, “Novice software developers, all over again,” in *Proc. ICER '08*, 2008, pp. 3–14.
- [3] “AI tutoring outperforms in-class active learning: an RCT introducing a novel research-based design in an authentic educational setting,” *Nature Scientific Reports*, vol. 15, 17458, 2025.
- [4] S. Kazemitabaar *et al.*, “Studying the effect of AI Code Generators on Supporting Novice Learners in Introductory Programming,” in *Proc. CHI '23*, 2023.
- [5] “A Systematic Review of Chatbots, Generative Tools, and Tutoring Systems in Programming Education,” *arXiv preprint*, arXiv:2510.03884, 2025.
- [6] “The Influence of Artificial Intelligence Tools on Learning Outcomes in Computer Programming: A Meta-Analysis,” *Computers*, vol. 14, no. 5, 185, 2025.
- [7] “AI-mediated feedback in gamified programming education: effects on vocational students’ achievement and motivation,” *Frontiers in Education*, vol. 11, 1846699, 2026.
- [8] “What the research shows about generative AI in tutoring,” Brookings Institution, 2025. [Online]. Available: <https://www.brookings.edu/articles/what-the-research-shows-about-generative-ai-in-tutoring/>